

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras. A CNN is trained on the CIFAR-10 dataset, and its performance is evaluated using accuracy, precision, recall, F1-score, confusion matrix, feature maps, and training-validation curves.

2. Learning Outcomes

After completing this experiment, the following learning outcomes were achieved:

1. Understanding the convolution operation and feature extraction.
2. Computing output dimensions using stride and padding.
3. Visualizing feature maps produced by convolution layers.
4. Comparing Max Pooling and Average Pooling.
5. Calculating the number of trainable parameters in CNN layers.
6. Building a CNN for multi-class image classification.
7. Evaluating a trained CNN using multiple performance metrics.
8. Analyzing overfitting using training and validation curves.

3. Background Theory

A Convolutional Neural Network is a deep learning architecture specifically designed to process grid-like data such as images. Unlike a fully connected network, a CNN learns local spatial patterns using convolution filters.

A typical CNN consists of the following layers:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Flatten Layer
- Fully Connected Layer
- Output Layer

The convolution operation can be represented as:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where:

- X represents the input image,
- K represents the convolution kernel or filter,
- Y represents the generated feature map.

The output spatial dimension of a convolution operation is:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where:

N	Input size
F	Filter size
P	Padding
S	Stride
O	Output size

4. Common Convolution Kernels

A convolution kernel is a small matrix that slides across an image to extract important visual features such as edges, textures, boundaries, and patterns.

4.1 Identity Kernel

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The identity kernel preserves the original image.

4.2 Sobel-X Kernel

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

This kernel is mainly used to detect vertical edges.

4.3 Sobel-Y Kernel

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

This kernel is mainly used to detect horizontal edges.

4.4 Laplacian Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

The Laplacian kernel detects edges in multiple directions.

4.5 Sharpening Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

This kernel emphasizes fine image details.

4.6 Box Blur Kernel

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

This kernel smooths an image by averaging neighbouring pixels.

4.7 Gaussian Blur Kernel

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Gaussian blur reduces noise while preserving the overall structure of an image.

4.8 Emboss Kernel

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

This kernel creates an embossed or three-dimensional appearance.

4.9 Outline Detection Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

This kernel highlights strong object boundaries.

4.10 Motion Blur Kernel

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

This kernel simulates diagonal motion blur.

5. Numerical Example: Convolution

Consider the input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and the kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The first output value is:

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

The second output value is:

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

The third output value is:

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

The fourth output value is:

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Therefore, the resulting feature map is:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

6. Numerical Example: Max Pooling

Consider the feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 Max Pooling operation:

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Hence, the pooled output is:

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

7. Numerical Example: Parameter Calculation

Consider an RGB input image of size:

$$32 \times 32 \times 3$$

Suppose a convolution layer has:

- Number of filters = 16
- Kernel size = 3×3

The number of parameters is:

$$(K_h \times K_w \times C_{in} + 1) \times C_{out}$$

Therefore:

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Hence, the convolution layer contains:

448 trainable parameters

8. Dataset Description

The CIFAR-10 dataset was used for this experiment.

Property	Value
Dataset	CIFAR-10
Training Images	50,000
Testing Images	10,000
Number of Classes	10
Image Size	$32 \times 32 \times 3$
Image Type	RGB Colour Images

The ten classes are:

- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

9. Experimental Procedure

The following procedure was followed:

1. The CIFAR-10 dataset was loaded using TensorFlow/Keras.
2. The images were normalized by dividing pixel values by 255.
3. Sample images from the dataset were displayed.
4. The class distribution was visualized.
5. A CNN model containing three convolution blocks was constructed.
6. Max Pooling was applied after each convolution layer.
7. A dense layer containing 128 neurons and ReLU activation was added.
8. Dropout was used before the final classification layer.
9. The model was trained using the Adam optimizer.
10. Sparse categorical cross-entropy was used as the loss function.
11. The model was trained for 20 epochs.
12. The trained model was evaluated on the independent test set.
13. Accuracy, precision, recall, F1-score, classification report, and confusion matrix were generated.
14. Feature maps from convolution layers were visualized.
15. Max Pooling and Average Pooling were compared experimentally.

10. CNN Model Architecture

The CNN architecture used in this experiment is:

Input $\rightarrow$ Conv2D(32) $\rightarrow$ Max Pooling $\rightarrow$ Conv2D(64) $\rightarrow$ Max Pooling $\rightarrow$ Conv2D(128) $\rightarrow$ Max Pooling
 $\rightarrow$ Flatten $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout $\rightarrow$ Dense(10, Softmax)

The architecture details are shown below.

Layer	Output Shape	Parameters
Conv2D	$32 \times 32 \times 32$	896
MaxPooling2D	$16 \times 16 \times 32$	0
Conv2D	$16 \times 16 \times 64$	18,496
MaxPooling2D	$8 \times 8 \times 64$	0
Conv2D	$8 \times 8 \times 128$	73,856
MaxPooling2D	$4 \times 4 \times 128$	0
Flatten	2048	0
Dense	128	262,272
Dropout	128	0
Dense	10	1,290
Total Trainable Parameters		356,810

11. Experimental Results and Plots

11.1 Sample Images from CIFAR-10

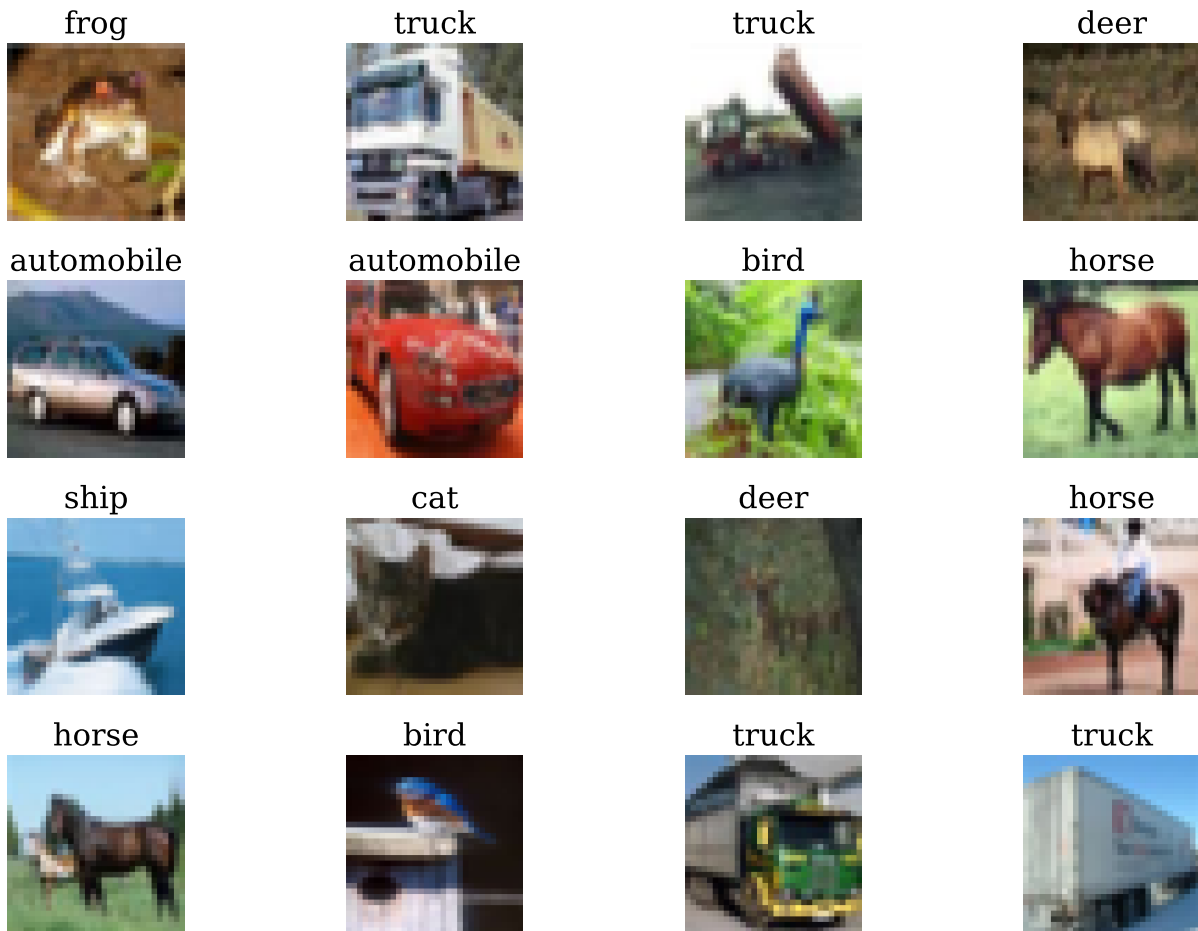


Figure 1: Sample Images from the CIFAR-10 Dataset

Inference:

The image provides samples of images found in the ten classes of CIFAR-10 dataset. The dataset is quite diverse visually as there are objects with various orientations, backgrounds, colors, and sizes.

11.2 Class Distribution

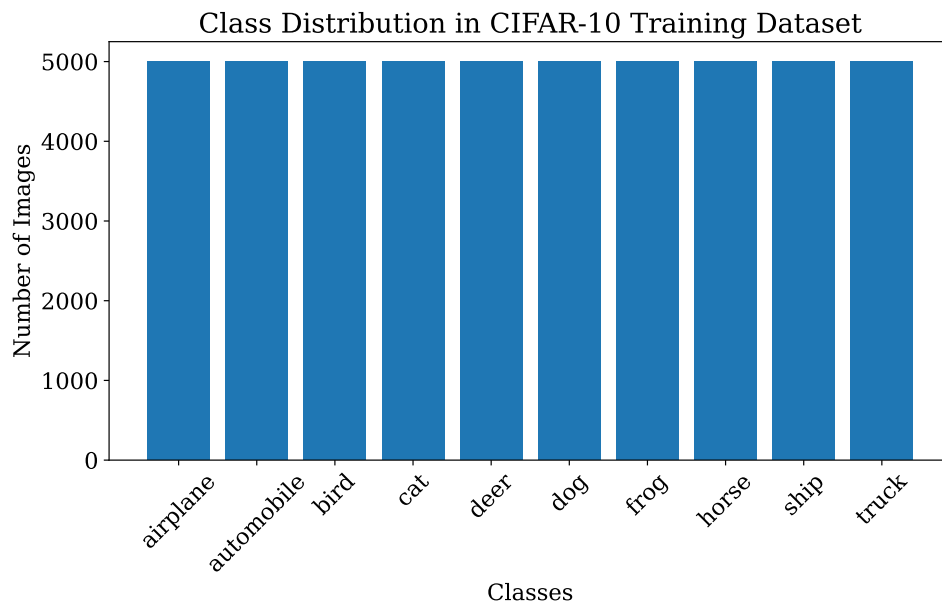


Figure 2: Class Distribution of the CIFAR-10 Dataset

Inference:

Class imbalance does not exist in the data since the number of images available for each class is about 5,000. This means that class imbalance should not lead to any bias towards a certain class during training.

11.3 Training and Validation Accuracy

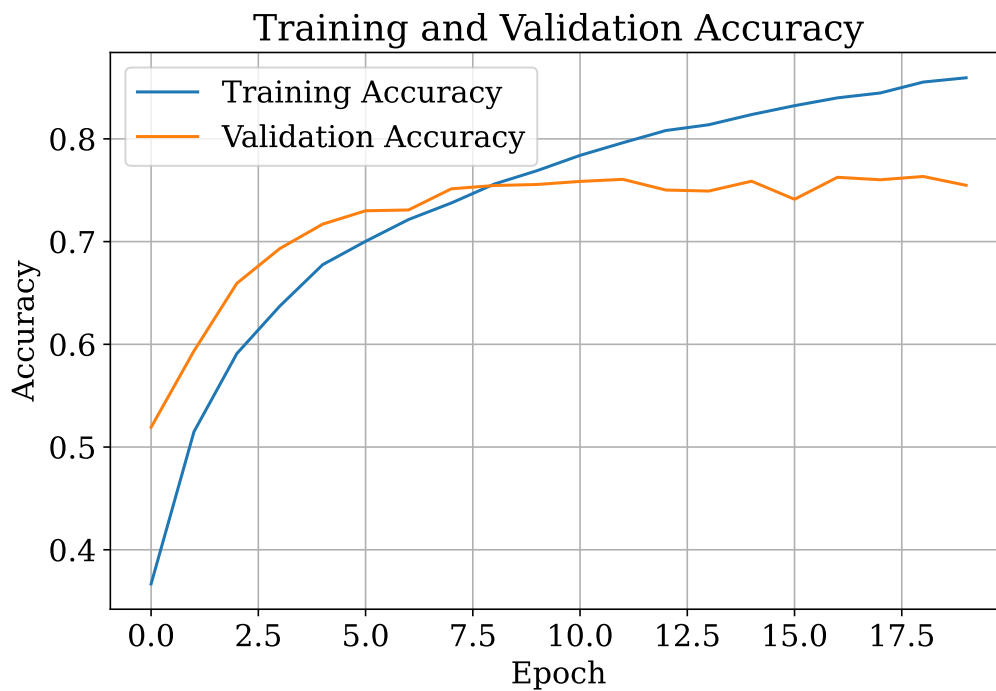


Figure 3: Training and Validation Accuracy versus Epoch

Inference:

Training accuracy steadily improved across the entire 20 epochs and became around 85.95% at the end epoch. On the other hand, validation accuracy rapidly increased in the early epochs but reached its peak at about 76.34% after which it fluctuated, showing signs of overfitting.

11.4 Training and Validation Loss

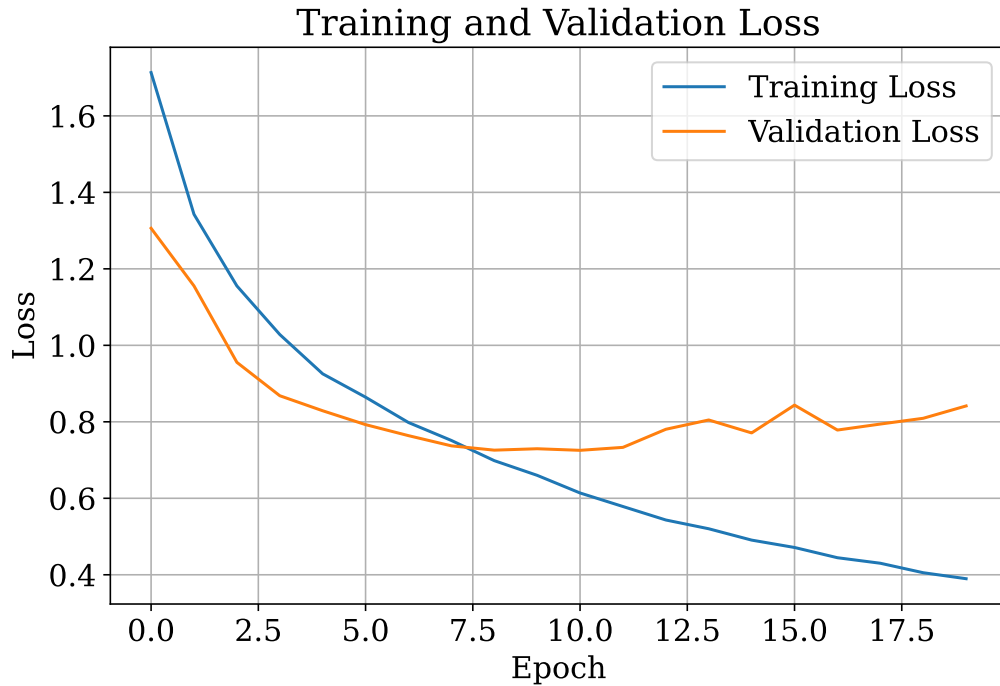


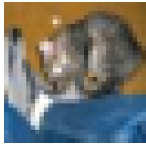
Figure 4: Training and Validation Loss versus Epoch

Inference:

The training loss was reducing consistently from about 1.71 to 0.39, and this shows that the CNN was able to learn from the training data. The validation loss, on the other hand, started reducing and then increased, thereby confirming overfitting.

11.5 Model Predictions

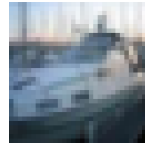
True: cat
Pred: cat



True: ship
Pred: ship



True: ship
Pred: airplane



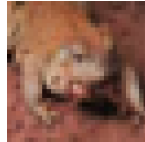
True: airplane
Pred: airplane



True: frog
Pred: frog



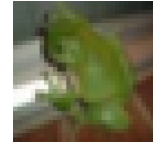
True: frog
Pred: frog



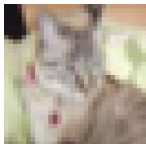
True: automobile
Pred: cat



True: frog
Pred: deer



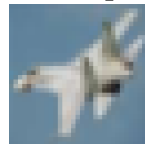
True: cat
Pred: cat



True: automobile
Pred: automobile



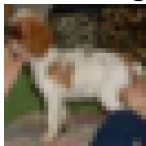
True: airplane
Pred: airplane



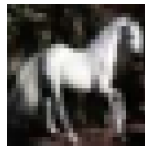
True: truck
Pred: truck



True: dog
Pred: dog



True: horse
Pred: horse



True: truck
Pred: truck



True: ship
Pred: ship

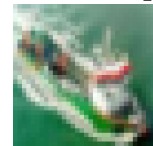


Figure 5: Sample Predictions Produced by the Trained CNN

Inference:

The prediction examples indicate that the CNN model can identify many objects regardless of differences in the backgrounds and objects. However, the wrong predictions happen when the objects belong to the same category.

11.6 Feature Map Visualization

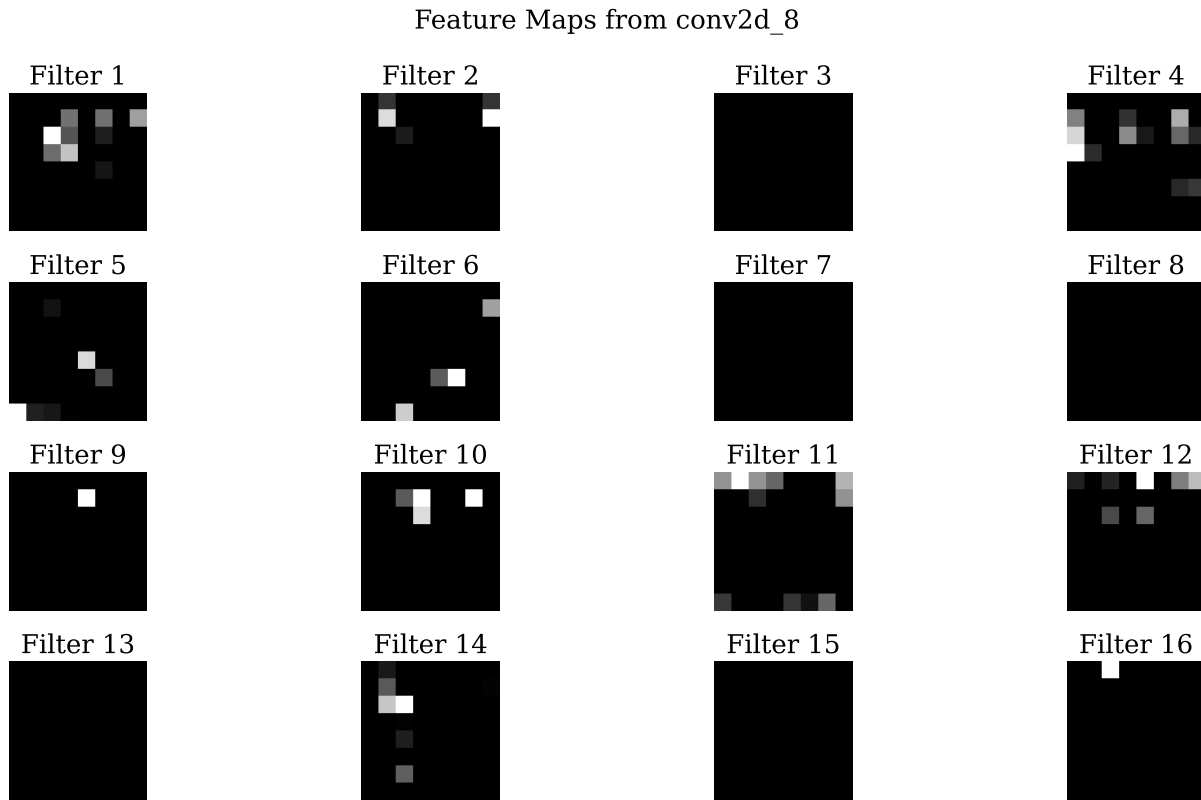


Figure 6: Feature Maps Extracted by the Convolutional Layers

Inference:

The feature maps show that different convolutional filters are learning different representations for the same input image. The early layers tend to extract low-level features such as edges and colors, whereas deep layers extract high-level object-related features. The observed feature map dimensions were:

Convolution Layer	Feature Map Shape
First Convolution Layer	$32 \times 32 \times 32$
Second Convolution Layer	$16 \times 16 \times 64$
Third Convolution Layer	$8 \times 8 \times 128$

11.7 Max Pooling versus Average Pooling

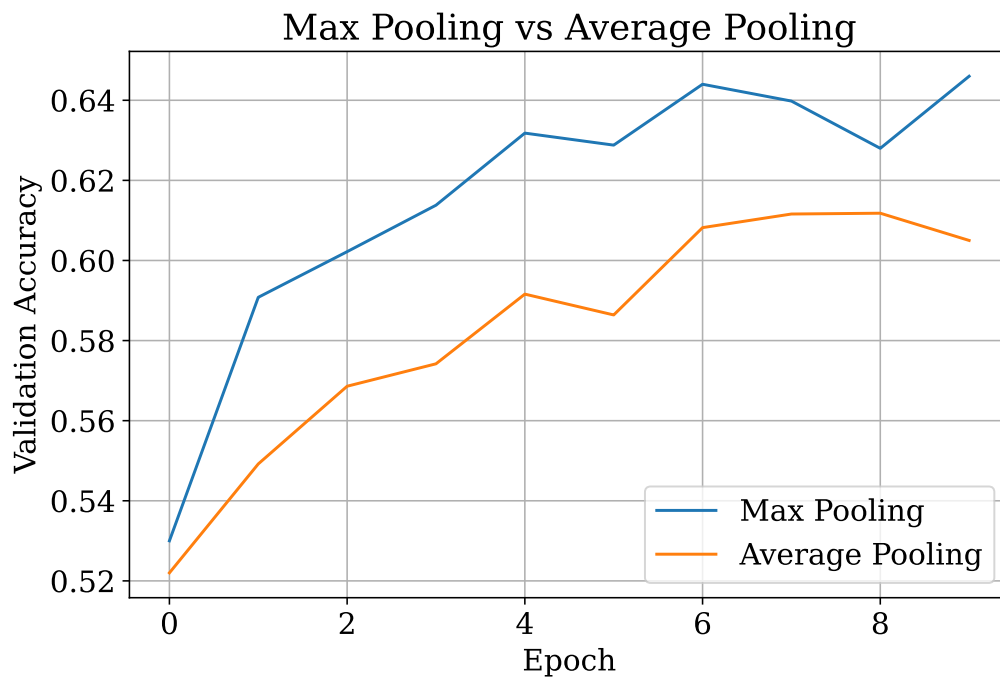


Figure 7: Comparison of Max Pooling and Average Pooling

Inference:

Max Pooling performed better in terms of validation accuracy when compared to Average Pooling in the experimental comparison. The maximum validation accuracy for Max Pooling was about 64.60%, while that for Average Pooling was 61.18%. Max Pooling retains the most activated neurons, and this seems to be more effective for this classification problem.

11.8 Confusion Matrix

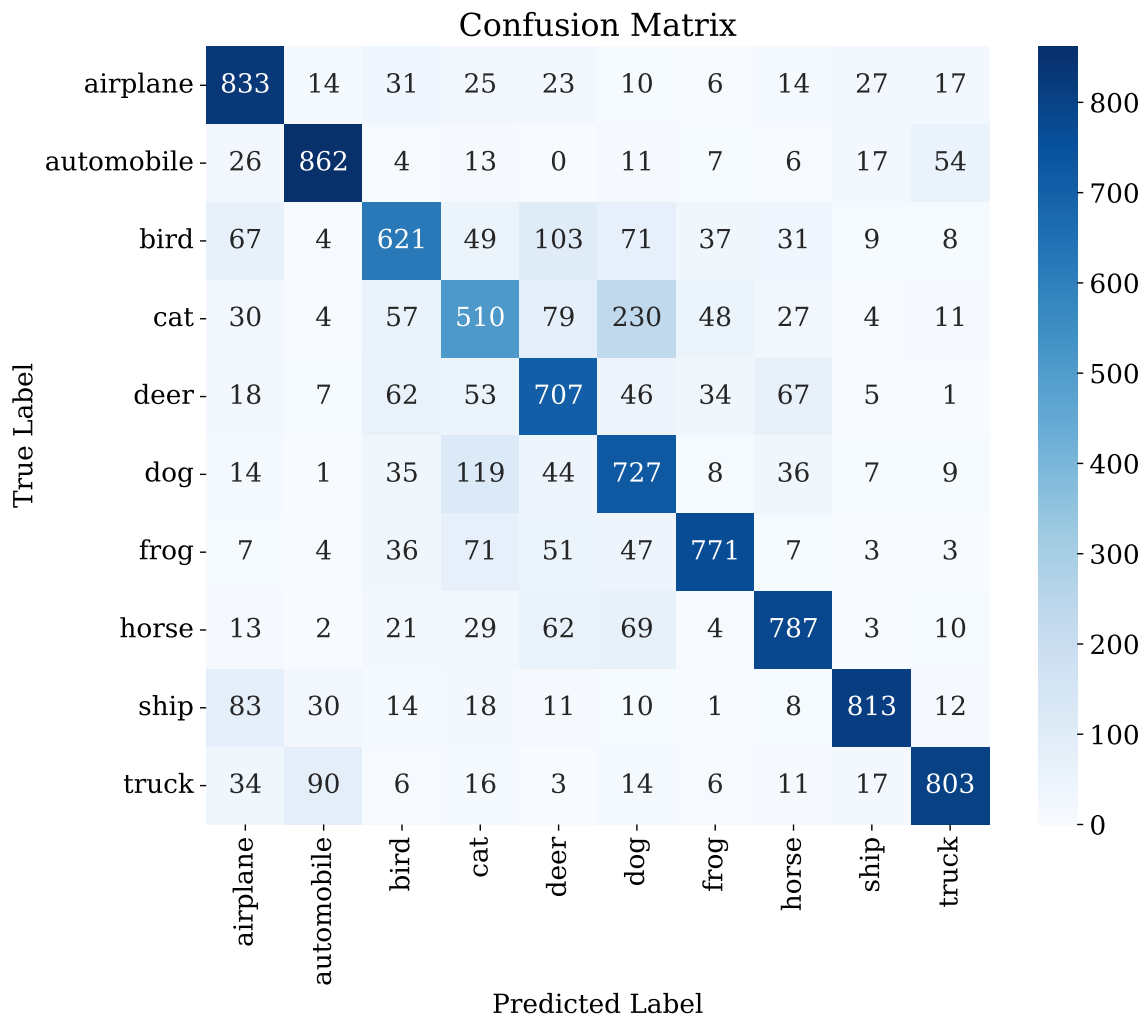


Figure 8: Confusion Matrix of the CNN on the CIFAR-10 Test Dataset

Inference:

The confusion matrix reveals a significant clustering of predictions on the main diagonal, which implies that a lot of the test samples have been classified correctly. The model exhibits good performance with respect to automobile, ship, truck, frog, and horse classifications, whereas confusions arise between cat, dog, bird, and deer classifications.

12. Training Results

The final epoch-wise training results are summarized below.

Epoch	Train Accuracy	Validation Accuracy	Train Loss	Validation Loss
1	36.68%	51.92%	1.7132	1.3061
2	51.50%	59.36%	1.3426	1.1554
3	59.10%	65.94%	1.1554	0.9555
4	63.72%	69.32%	1.0283	0.8682
5	67.75%	71.70%	0.9255	0.8289
6	70.03%	73.00%	0.8644	0.7926
7	72.14%	73.08%	0.7982	0.7638
8	73.76%	75.14%	0.7512	0.7371
9	75.60%	75.46%	0.6985	0.7259
10	76.91%	75.56%	0.6600	0.7296
11	78.39%	75.86%	0.6139	0.7254
12	79.63%	76.06%	0.5785	0.7332
13	80.81%	75.02%	0.5432	0.7805
14	81.38%	74.92%	0.5203	0.8048
15	82.37%	75.88%	0.4907	0.7709
16	83.23%	74.12%	0.4714	0.8436
17	83.99%	76.26%	0.4446	0.7784
18	84.47%	76.02%	0.4302	0.7941
19	85.53%	76.34%	0.4054	0.8092
20	85.95%	75.48%	0.3897	0.8415

The best validation accuracy was obtained at Epoch 19 with approximately 76.34%. The final epoch produced higher training accuracy but lower validation accuracy, further indicating that continued training increased overfitting.

13. Final Test Set Performance

The trained CNN was evaluated on the independent CIFAR-10 test dataset containing 10,000 images.

Metric	Value
Test Loss	0.8892
Test Accuracy	74.34%
Weighted Precision	74.85%
Weighted Recall	74.34%
Weighted F1-Score	74.40%
Trainable Parameters	356,810

14. Class-wise Classification Performance

Class	Precision	Recall	F1-Score
Airplane	0.74	0.83	0.78
Automobile	0.85	0.86	0.85
Bird	0.70	0.62	0.66
Cat	0.56	0.51	0.54
Deer	0.65	0.71	0.68
Dog	0.59	0.73	0.65
Frog	0.84	0.77	0.80
Horse	0.79	0.79	0.79
Ship	0.90	0.81	0.85
Truck	0.87	0.80	0.83

The highest overall performance class-wise was shown by automobile and ship, both showing F1-score value of approximately 0.85. The worst performing class was that of cat, which showed F1-score value of approximately 0.54. This could be because of similarity between images of cats and dogs in CIFAR-10 dataset.

15. Comparison of Pooling Methods

A separate experiment was conducted to compare Max Pooling and Average Pooling.

Pooling Method	Best Validation Accuracy
Max Pooling	64.60%
Average Pooling	61.18%

From the above results, it can be seen that Max Pooling worked better in this experiment. While Max Pooling keeps the best feature response in each local area, Average Pooling averages the responses.

16. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolutional neural networks use local receptive fields and feature sharing. As a result, they have much fewer parameters and can learn spatial features.

2. How does stride affect the feature map size?

Stride is increased to reduce the size of the spatial dimensions of the output feature map. An increased stride will reduce computations but may lead to loss of spatial information.

3. What is the role of padding?

Padding determines the size of the output feature map in space. Same padding assists in maintaining the size of the image, while valid padding decreases it.

4. Why is pooling used?

Pooling decreases the dimensions of feature maps in space, cuts down on computation, and offers tolerance to translation in the image.

5. How do feature maps represent image characteristics?

The filters learn various patterns. The first layers tend to identify simple patterns such as edges and color changes, but the later layers identify complex patterns.

6. Why do CNNs require fewer parameters than MLPs for images?

The CNN makes use of local connections and weight sharing. This is because the filter is used in different positions of the images, leading to a reduction in the number of parameters from that of a fully connected model.

7. Why did the model show signs of overfitting?

While the training accuracy kept rising to 85.95%, the validation accuracy was about 75-76%. Moreover, the validation loss was also rising in the late epochs, which is evidence of the growing specialization of the model on the training set.

8. Which pooling method performed better?

Max Pooling outperformed Average Pooling in the conducted experiment. The highest validation accuracy of Max Pooling was about 64.60%, while that of Average Pooling was about 61.18%.

17. Conclusion

For this experiment, a Convolutional Neural Network was implemented and trained for CIFAR-10 image classification. This experiment showed how to do the basic operations of CNNs such as convolution, feature extraction, pooling, parameters calculation and multi-class classification.

The CNN implemented for this experiment had the following metrics: Test accuracy of **74.34%** Weighted precision of **74.85%** Weighted recall of **74.34%** Weighted F1-score of **74.40%** It also included **356,810 trainable parameters**.

For training and validation curves, we could see how the learning process took place in the beginning, and later on there was some overfitting. For feature-map visualization, we could observe how the convolution layers learn visual features. In the case of pooling, it was shown that Max Pooling works better than Average Pooling.

18. References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson.
4. TensorFlow Documentation: <https://www.tensorflow.org>
5. Keras Documentation: <https://keras.io>
6. CIFAR-10 Dataset: <https://www.cs.toronto.edu/~kriz/cifar.html>